End-to-End LLM Pipeline for Tourism Assistant

Personal Learning Project — Technical Report

Made by	Younes AIT SI ABBOU
Date	April 29, 2026
Version	1.0
Status	Learning / Noncommercial

**This document is a personal learning report and does not represent any commercial product or organization. **

Table of Contents

Introduction 4
Project Links 4
Technology Stack at a Glance 5
Part 1 — Dataset Generation Pipeline 6
1.0 Pipeline Overview 6
1.1 Data Source — Reddit Scraping 7
Source Subreddits and Collection Volume 7
Risks and Known Biases 7
1.2 Data Cleaning and Filtering 8
1.3 Structuring — CSV to JSON with Metadata Enrichment 8
1.4 Chunking Strategy 9
1.5 Embeddings 9
1.6 Qdrant — Vector Storage 9
1.7 RAG Retrieval Pipeline 9
1.8 Dataset Generation — Core Stage 10
1.9 Question Generation and Diversity 10
1.10 Output Quality Assessment 11
Part 2 — Design Decisions and Architecture Rationale 12
2.1 Why Raw Reddit Data Was Not Used Directly for Training 12
2.2 The Complementary Roles of RAG and Fine-Tuning 12
Part 3 — Fine-Tuning 13
3.1 Model Selection — Qwen2-1.5B-Instruct 13
3.2 Compute Environment 13
3.3 Training Approach — QLoRA Failure and FP16 LoRA 13
3.4 Final Training Configuration 14
3.5 Training Metrics 14
3.6 Qualitative Training Observations 15
Part 4 — Deployment Architecture 16
4.1 End-to-End Inference Flow 16
4.2 Deployment Component Summary 16

Part 5 — Limitations 18
5.1 Limitations Summary 18
Part 6 — Challenges Encountered 19
Part 7 — Results 21
7.1 Results Summary 21
Part 8 — Future Improvements 22
Conclusion 23

Introduction

This report documents a self-initiated personal project undertaken to gain practical, end-to-end experience building an AI system powered by Large Language Models (LLMs). The project covers the full pipeline from raw data acquisition through retrieval-augmented generation (RAG), dataset engineering, supervised fine-tuning, and a lightweight deployment stack.

The domain chosen is tourism assistance — a use case that offers a naturally rich question-and-answer surface while remaining tractable for a single developer working within the constraints of free-tier cloud compute.

It is important to state clearly at the outset: this is a learning-focused implementation. The system is not production-ready, not commercially deployed, and not affiliated with any organisation. Accuracy is limited, the training dataset is small, data verification was incomplete, and model outputs can be incorrect or inconsistent.

Project Links

Resource	Link
GitHub Repository	Link
Hugging Face Model	Link
Hugging Face Dataset	Link
Demo (Gradio UI)	Link

Table: Project artefact references

Technology Stack at a Glance

Layer	Technology	Notes
Data collection	PRAW (Reddit API)	Posts + comments from travel subreddits
Data cleaning	Node.js, regex, langdetect	Score filter, dedup, encoding fix
Embedding model	all-MiniLM-L6-v2	384-dim vectors, CPU-friendly
Vector store	Qdrant Cloud (free tier)	~1,200 vectors, cosine similarity
Dataset gen. LLM	Qwen2.5-7B via Ollama (local)	Grounded Q&A pair generation
Fine-tune base	Qwen2-1.5B-Instruct	LoRA adapters, FP16
Training stack	transformers, peft, trl, accel.	Kaggle T4 GPU (free tier)
Backend	Python	Embedding, retrieval, prompt assembly
Frontend	Gradio (Hugging Face Spaces)	Conversational UI
Model hosting	Hugging Face Inference Endpoints	Merged LoRA + base weights

Table: Full technology stack summary

Part 1 — Dataset Generation Pipeline

The dataset generation pipeline is the foundation of the entire project. Its quality directly determines every downstream result, from retrieval accuracy to fine-tuning behaviour. This section documents each stage of the pipeline in detail, including the rationale for design decisions, trade-offs made, and known weaknesses.

1.0 Pipeline Overview

The diagram below shows the sequential stages of the dataset generation pipeline. Each stage feeds the next; failures or noise introduced at any stage propagate downstream.

Dataset Generation Pipeline
Step 1 — Data Collection Reddit posts & comments via PRAW, score-filtered, English only
Step 2 — Data Cleaning HTML strip, deduplication, length filter, encoding fix
Step 3 — Structuring CSV → JSON with metadata enrichment (topic tags, quality score, URL)
Step 4 — Chunking Whole-comment chunks; posts > 600 tokens split with 50-token overlap
Step 5 — Embedding all-MiniLM-L6-v2 → 384-dim dense vectors, batch CPU inference
Step 6 — Indexing Vectors + payload stored in Qdrant Cloud (cosine metric)
Step 7 — RAG Retrieval Query → embed → top-5 Qdrant search → context block assembly
Step 8 — Dataset Generation Ollama Qwen2.5-7B: context + prompt → grounded Q&A pair (JSONL)
Step 9 — Post-processing Strip artefacts, enforce ChatML format, manual quality sample

Table: Sequential stages of the dataset generation pipeline

1.1 Data Source — Reddit Scraping

Reddit was selected as the primary data source for several practical reasons. The platform hosts a large volume of organic, first-person travel discussions across subreddits such as r/travel, r/solotravel, r/Morocco, r/backpacking, and various destination-specific communities. The conversational structure of Reddit threads maps naturally onto the question-answer format required for fine-tuning, and the PRAW library provides structured, rate-limited access without paid tiers.

Source Subreddits and Collection Volume

Subreddit	Approx. Posts Scraped	Primary Topic Focus
r/travel	~300	General international travel advice
r/Morocco	~250	Morocco-specific tips, logistics, culture
r/solotravel	~200	Solo travel safety, planning, logistics
r/backpacking	~150	Budget travel, accommodation, routes
r/travel (comments)	~400	Top-voted answers to travel questions

Table: Reddit sources and approximate scraping volumes

Risks and Known Biases

Reddit data carries significant noise and bias. The user base is predominantly English-speaking, Western, and younger; highly upvoted comments may be opinion rather than fact; posts from prior years may contain outdated travel conditions; and scraping was bounded by API rate limits, resulting in uneven destination coverage. These risks were accepted as tolerable for a learning project but would be disqualifying for a deployed system.

1.2 Data Cleaning and Filtering

Raw scraped data was passed through a multi-stage cleaning pipeline. The table below summarises each filter, its threshold, and the trade-off involved.

Filter	Threshold / Rule	Trade-off
Minimum length	> 80 characters	Removes one-liners; some useful short answers discarded
Score threshold	Comment score >= 5	Weak quality proxy; incorrect high-voted advice passes through
Deduplication	Exact hash match	Near-duplicates not detected; semantic redundancy remains
HTML / MD strip	Regex-based	Edge cases (nested quotes, tables) produce malformed text
Language detection	ASCII ratio + stopwords	Imprecise; some non-English slips through, some valid EN rejected

Table: Data cleaning filters with thresholds and trade-offs

1.3 Structuring — CSV to JSON with Metadata Enrichment

Cleaned records were initially stored in CSV for ease of inspection, then converted to JSON to accommodate nested metadata. Each document record was enriched with the following fields:

topic_tags	Keyword-matched topic labels (visa, accommodation, transport, safety). ~60% accuracy — not rigorously validated.
source_url	Canonical Reddit permalink retained for traceability and manual review.
quality_score	Composite heuristic: normalised blend of comment score, post score, and text length (0–1 range). Not externally validated.
chunk_id	Deterministic UUID derived from source URL + character offset. Used to prevent duplicate indexing on re-runs.

Table: Metadata fields added during JSON enrichment

1.4 Chunking Strategy

Individual Reddit comments were treated as atomic chunks. Post bodies exceeding 600 tokens were split using a sliding window with 50-token overlap. The 600-token threshold fits within the embedding model context window while avoiding information loss.

1.5 Embeddings

The all-MiniLM-L6-v2 model from sentence-transformers was used to generate dense vector embeddings. The selection rationale is summarised below.

Model	all-MiniLM-L6-v2 (sentence-transformers)
Output dims	384-dimensional dense vector
Parameters	~22 million
Inference	CPU, batched — no GPU required for indexing phase
Strength	Fast, offline, competitive on general English semantic similarity benchmarks
Weakness	Not domain-adapted for tourism; specialised terminology may not cluster accurately

Table: Embedding model properties and trade-offs

1.6 Qdrant — Vector Storage

Qdrant was selected after evaluating FAISS, Chroma, and Weaviate. The key differentiators were a managed free-tier cloud cluster, dual REST and gRPC interfaces for cross-language compatibility, and native payload filtering for hybrid retrieval without a separate metadata database.

1.7 RAG Retrieval Pipeline

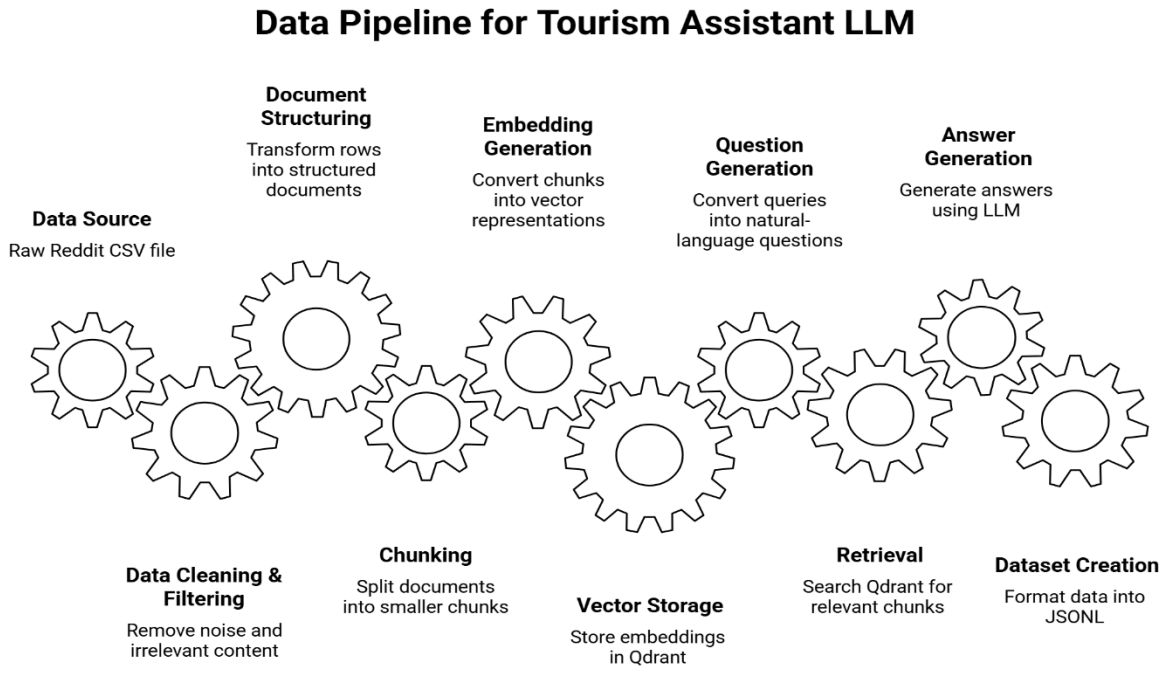
The retrieval pipeline follows a standard dense retrieval pattern: the user query is embedded with the same model used at index time; a top-k approximate nearest-neighbor search is issued against Qdrant; retrieved chunk texts are assembled into a context block prepended to the prompt.

Step	Action	Detail
1	**Query embedding**	Same all-MiniLM-L6-v2 model — consistency with index-time embeddings is critical
2	**Top-k ANN search**	k = 5 (default); k = 10 tested but no quality improvement observed
3	**Score ranking**	Chunks ranked by cosine similarity; threshold filtering not applied
4	**Context assembly**	Top-k texts concatenated into delimited block prepended to prompt
5	**Prompt injection**	Format: "Context: [...] Question: [...] Answer:"

Table: RAG retrieval pipeline steps

1.8 Dataset Generation — Core Stage

A locally hosted Qwen2.5-7B instance (via Ollama) was used to generate question-answer pairs from retrieved context chunks. Each generation request provided a context chunk and instructed the model to produce one realistic tourism question answerable from that context, followed by a grounded answer citing only information present in the provided text. This grounding mechanism reduces hallucination in training examples relative to context-free generation, though it does not guarantee factual correctness of the source Reddit material itself.



1.9 Question Generation and Diversity

Temperature 0.7 was used to encourage lexical variation. The pipeline targeted four question types:

Question Type	Example Pattern
Factual recall	"What documents are required for a visa to Morocco?"
Comparative	"How does train travel compare to bus travel for this route?"
Advisory	"What precautions should a solo traveller take in this area?"
Practical logistics	"How long does this journey typically take?"

Table: Targeted question types and example patterns

Despite these efforts, the resulting question set skewed toward factual recall, which is the mode the generative model defaulted to most frequently when given Reddit-style context.

1.10 Output Quality Assessment

A manually reviewed sample of approximately 40 examples was used to estimate generation quality.

Quality Category	Estimated %	Description
Acceptable	~65%	Grounded, coherent, useful answer
Borderline	~25%	Vague or weakly grounded; included but degrade training quality
Rejected / Empty	~10%	Factually incorrect, empty, or malformed

Table: Manual quality assessment of generated Q&A samples ($n \approx 40$)

No automated quality filter was applied to the full dataset. A non-trivial proportion of training examples therefore contain low-quality content.

Part 2 — Design Decisions and Architecture Rationale

This section explains the key architectural decisions, their reasoning, and the trade-offs involved.

2.1 Why Raw Reddit Data Was Not Used Directly for Training

Raw Reddit text is unstructured, inconsistently formatted, and contains a high proportion of content not useful as a training signal. Direct fine-tuning on raw prose does not create the instruction-following behaviour needed for a question-answering assistant. Constructing (instruction, input, output) triples manually at the scale needed was not feasible without a generation intermediary.

2.2 The Complementary Roles of RAG and Fine-Tuning

A critical design insight governs the entire architecture: RAG and fine-tuning are not interchangeable mechanisms. They address different problems and are most effective when used together.

Dimension	RAG	Fine-Tuning
Primary role	Provides external knowledge	Shapes response behaviour and style
Mechanism	Retrieves relevant context at inference time	Adjusts model weights via gradient updates
Knowledge update	Update vector store; no retraining needed	Requires new training run for each update
Hallucination risk	Reduced when retrieval is accurate	Not reduced without grounded training data
Compute cost	Index time + query time embedding	GPU hours; proportional to dataset size
Failure mode	Retrieval miss → no grounding for answer	Overfitting on small dataset; memorisation
Used at	Dataset generation time + inference time	Offline training phase only

Table: Comparison of RAG and fine-tuning across key dimensions

Fine-tuning on 150–200 examples cannot inject meaningful new factual knowledge into a 1.5B parameter model — it adjusts behavioural style. This is why the RAG component remains necessary at inference time even after fine-tuning.

Part 3 — Fine-Tuning

3.1 Model Selection — Qwen2-1.5B-Instruct

Qwen2-1.5B-Instruct was selected based on compute constraints rather than optimal performance. At 1.5B parameters it fits in a single T4 GPU without quantisation for inference, and the instruction-tuned variant already exhibits basic Q&A behaviour that fine-tuning can refine.

3.2 Compute Environment

Platform	Kaggle (free tier)
GPU	NVIDIA T4 — 15 GB VRAM
Session limit	9 hours per run
Constraint impact	Bounded epochs, dataset size, and hyperparameter search iterations

Table: Training compute environment

3.3 Training Approach — QLoRA Failure and FP16 LoRA

The initial configuration attempted QLoRA (4-bit NF4 quantisation via bitsandbytes). This failed with CUDA kernel errors during the forward pass — non-deterministically reproducible and traced to a bitsandbytes / CUDA driver version incompatibility in the Kaggle environment. After exhausting debugging time, the approach was abandoned in favour of standard FP16 LoRA.

3.4 Final Training Configuration

Hyperparameter	Value
Precision	FP16
LoRA rank (r)	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, v_proj
Epochs	3
Batch size	4 (grad. accum. steps = 4 → effective BS = 16)
Learning rate	2e-4 with cosine annealing schedule
Warmup ratio	0.03
Max sequence length	512 tokens
Training examples	~150–200 samples (90 / 10 train / val split)
Framework	transformers + peft + trl (SFTTrainer) + accelerate

Table: Final fine-tuning hyperparameters and configuration

3.5 Training Metrics

Epoch	Train Loss	Val Loss	Notes
1	~1.62	~1.74	Rapid initial descent
2	~1.10	~1.21	Continued improvement; val tracking train
3	~0.85	~1.29	Mild divergence — overfitting indicator on small val set

Table: Approximate training and validation loss per epoch

Final validation perplexity was approximately 2.3. This metric should be interpreted with caution given the validation set comprised only 15–20 examples — too small for statistically meaningful conclusions.

3.6 Qualitative Training Observations

Manual inspection of outputs revealed improved response style and domain vocabulary after fine-tuning, with no meaningful improvement in factual accuracy on out-of-distribution queries. Some memorisation of training examples was observed, as expected with a small dataset.

Part 4 — Deployment Architecture

The deployment stack was designed to be minimal, functional, and compatible with free or low-cost hosting tiers.

4.1 End-to-End Inference Flow

Step	Component	Action
1	**Gradio UI (HF Spaces)**	User submits natural language tourism query
2	**Node.js / Express backend**	Receives query; triggers embedding subprocess
3	**Embedding service (MiniLM)**	Encodes query to 384-dim vector
4	**Qdrant Cloud**	Top-5 ANN search; returns chunk texts + similarity scores
5	**Backend — context assembly**	Formats retrieved chunks + query into prompt template
6	**HF Inference Endpoint (Qwen2-1.5B)**	Generates response; streams tokens back
7	**Gradio UI**	Displays response in chat window

Table: End-to-end inference flow from user query to response

4.2 Deployment Component Summary

Hugging Face Inference Endpoint	Hosts merged LoRA + base model. Managed serving, free-tier throughput limits apply.
Gradio (Hugging Face Spaces)	Conversational chat UI. Streaming mode disabled due to visual artefacts; synchronous mode used.
Node.js / Express backend	Middleware for embedding, retrieval, and prompt assembly. Python subprocess used for embedding (cross-language overhead noted as a limitation).
Qdrant Cloud (free tier)	Production vector store. REST client, API key in environment variable. ~1,200 vectors, cosine metric.

Table: Deployment component roles and notes

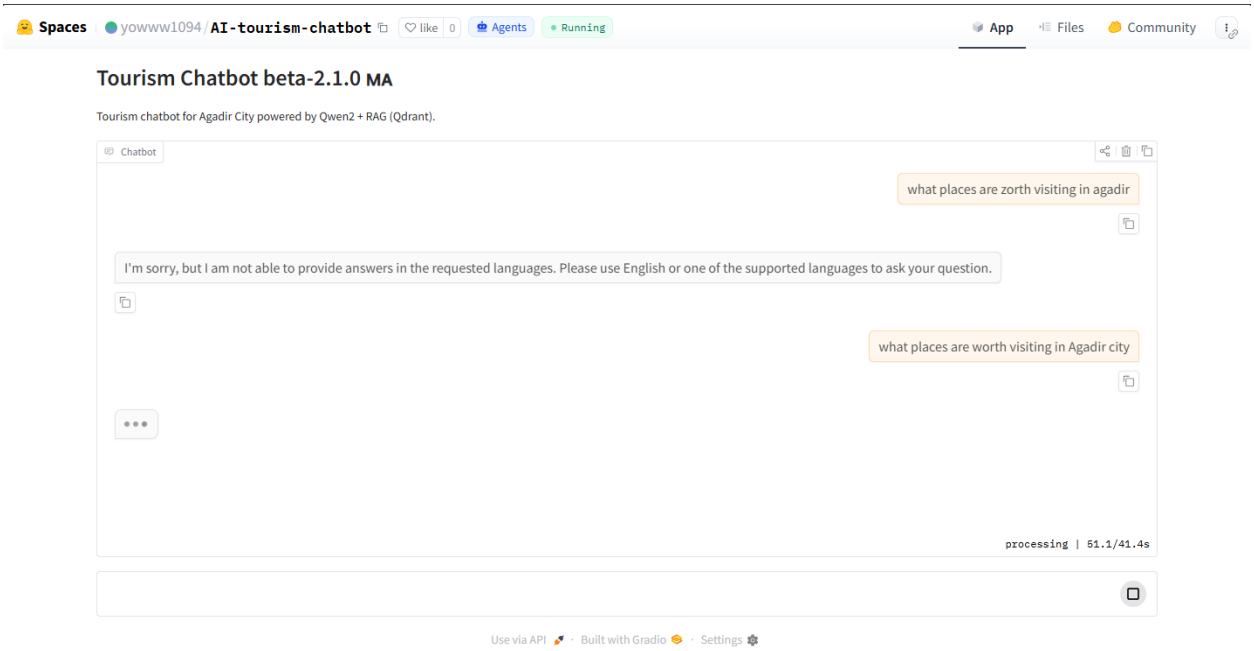


Figure: Chatbot interface inside Hugging Face

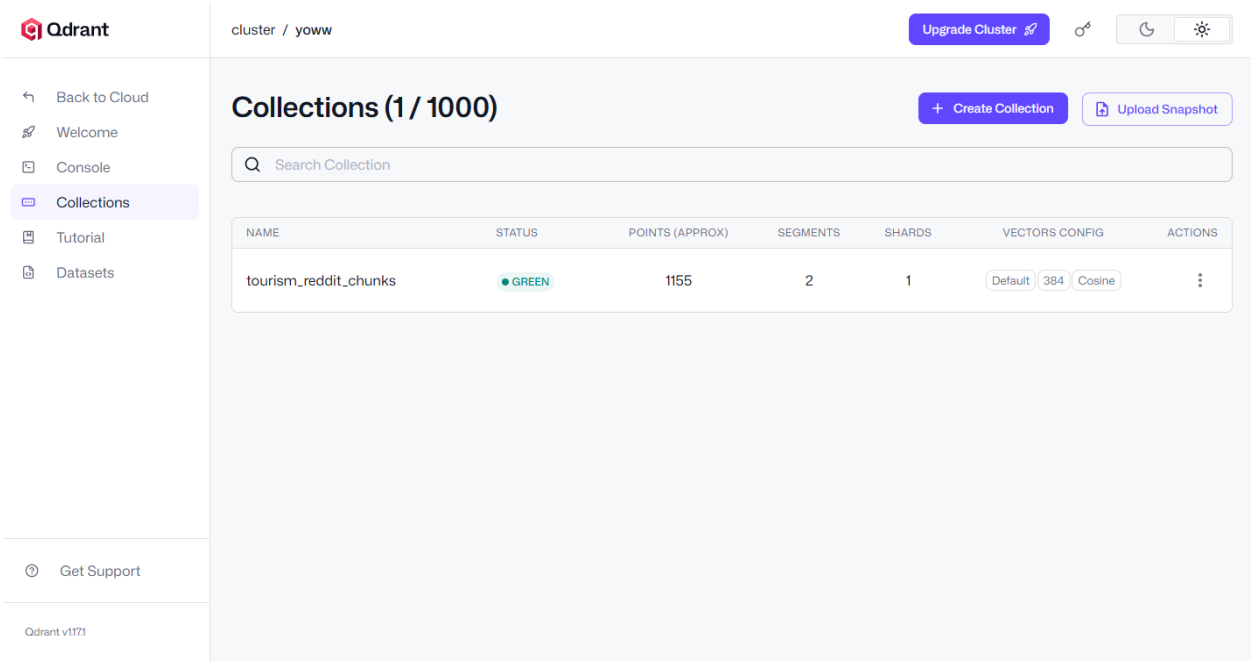


Figure: Qdrant Collection Interface

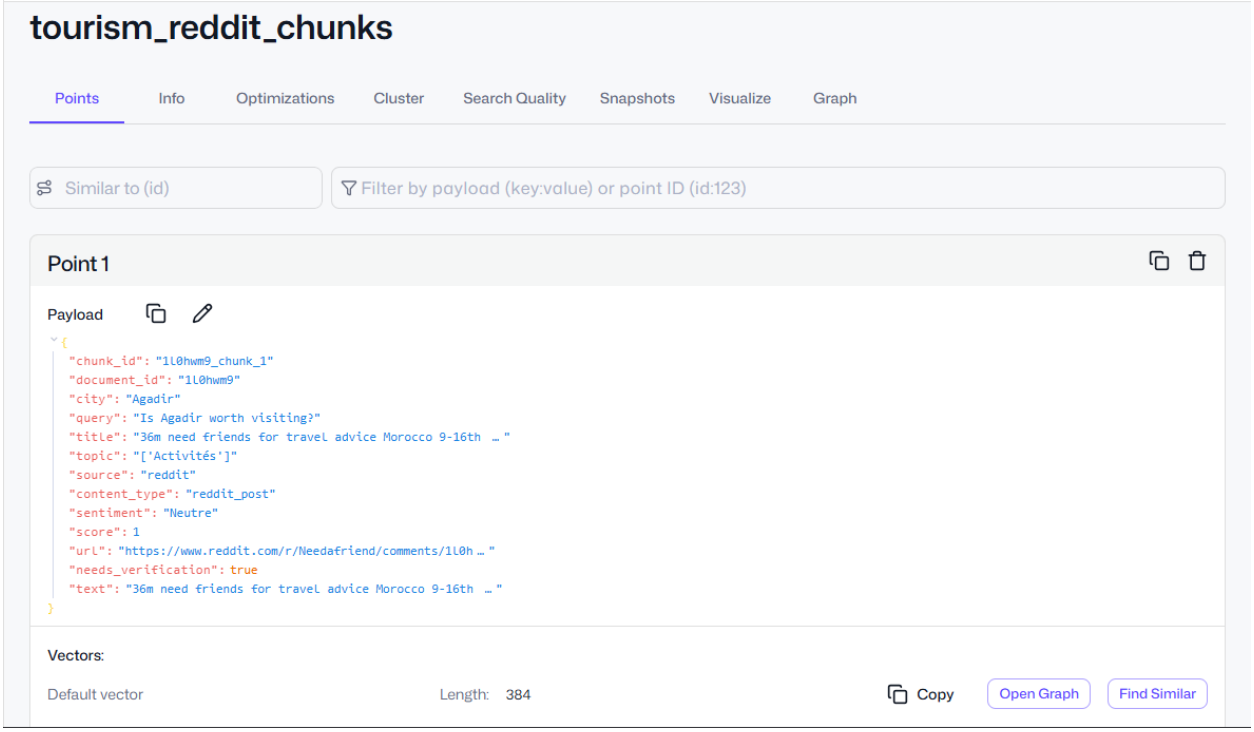


Figure: Qdrant collection chunk example

Part 5 — Limitations

This section provides an honest account of the system's limitations. These are not incidental — many are inherent to the scale and scope of a single-developer learning project.

5.1 Limitations Summary

Category	Limitation	Impact
Dataset	Only ~1,200 chunks; ~150–200 training examples	Weak generalisation; large coverage gaps
	Manual validation on ~40 samples only	Unknown proportion of bad training data
	Incomplete metadata tagging (~60% accuracy)	Unreliable retrieval filtering
	No data versioning	Experiments not fully reproducible

Category	Limitation	Impact
Retrieval	Non-domain-adapted embedding model	Vocabulary mismatch on specialist queries
	Fixed k=5; no re-ranking	Sub-optimal context for many query types
	No hybrid BM25 + dense search	Exact-match queries underserved
Model	Hallucination of specific facts (dates, prices)	Unacceptable for factual assistant
	No answer abstention mechanism	Produces plausible-sounding wrong answers
	Response inconsistency across runs	Same query may produce different answers
System	8–12 second end-to-end latency	Not suitable for real user-facing deployment
	HF endpoint cold-start timeouts	First query after idle period may fail

Table: Consolidated limitations by category

Part 6 — Challenges Encountered

The following challenges were the primary sources of unplanned time expenditure and engineering learning during the project.

Challenge	Description and Resolution
GPU / session time limits	Kaggle's 9-hour session cap ruled out longer runs. Solved by designing experiments to complete within a session and using checkpoint saving.
QLoRA / bitsandbytes failure	Non-deterministic CUDA errors during quantised forward pass. Root cause: bitsandbytes / CUDA driver version mismatch in Kaggle environment. Resolution: switched to FP16 LoRA.
Dataset formatting corruption	Character encoding mismatch between PRAW response and JSON serialiser introduced non-UTF-8 bytes, causing tokeniser silent failures. Detected via training set size discrepancy.

Challenge	Description and Resolution
Gradio streaming artefacts	Partial tokens rendered as separate chat bubbles. Resolution: disabled streaming, accepted higher perceived latency.
Qdrant API version drift	Community code examples used deprecated API patterns causing silent failures (empty results with no error). Resolution: strict adherence to current client library documentation.
Hugging Face auth complexity	Multiple token types (user, write, endpoint service account) with unclear documentation. Endpoint cold-starts caused first-request timeouts requiring client-side retry logic.
Ollama local inference overhead	Qwen2.5-7B occupied most system RAM during dataset generation, preventing parallel tasks. ~15 tokens/sec throughput meant 200 examples required several hours.

Table: Challenges, descriptions, and resolutions

Part 7 — Results

Results are reported qualitatively. The dataset and evaluation setup do not support rigorous quantitative benchmarking.

7.1 Results Summary

Aspect	Outcome	Notes
Response style and tone	**Improved**	More focused, helpful register vs. base model
Domain vocabulary usage	**Improved**	More consistent use of tourism terminology
RAG + FT combined vs. either alone	**Improved**	Better in-distribution answers when both active
End-to-end pipeline completeness	**Achieved**	Full scrape-to-deploy pipeline functional
Factual accuracy (out-of-distribution)	**No change**	Hallucination rate comparable to base model
Retrieval precision (all query types)	**Partial**	Good for in-vocab queries; poor for specialist terms
Coverage across travel domain	**Poor**	Large topic gaps due to small corpus
Response latency	**Poor**	8–12 seconds; not suitable for live deployment
Evaluation rigour	**Poor**	Informal only; no formal benchmark applied

Table: Qualitative results summary

The system as built would not be suitable for deployment as a real tourist information service. Its value is entirely pedagogical: it demonstrates that the full pipeline is assembleable by a single developer with open-source tools and free compute, and makes trade-offs concrete through direct experimentation.

Part 8 — Future Improvements

Improvements are ordered by estimated impact on overall system quality.

Pri.	Improvement	Estimated Impact	Key Actions
1	**Larger, higher-quality dataset**	High	Target 2,000–5,000 verified Q&A pairs from authoritative sources; add automated quality filter
2	**Domain-adapted embedding model**	High	Replace MiniLM with E5-large-v2 or BGE-large; consider fine-tuned tourism embedding
3	**Cross-encoder re-ranking**	Medium	Add re-ranking step with cross-encoder/ms-marco model after initial ANN retrieval
4	**Hybrid BM25 + dense retrieval**	Medium	Add sparse BM25 index for exact-match queries; merge rankings with RRF
5	**Larger base model (7B)**	Medium	Qwen2-7B-Instruct or equivalent; requires stable QLoRA setup or higher-VRAM GPU
6	**Formal evaluation framework**	Medium	Held-out test set (100+ verified examples); ROUGE, BERTScore, human preference ratings
7	**Multilingual support**	Low–Med	Multilingual embeddings + base model; French, Arabic, Darija, Spanish data required
8	**Production infrastructure**	Low	Containerised services, message queue, caching, rate limiting, monitoring
9	**UI improvements**	Low	Source citation display, feedback mechanism, mobile-responsive design

Table: Future improvements ordered by estimated impact

Conclusion

This report has documented the design, implementation, and evaluation of an end-to-end LLM pipeline for a tourism assistant, built as a personal learning project. The system integrates Reddit data scraping, vector-based retrieval-augmented generation, locally grounded dataset generation, supervised fine-tuning with LoRA, and a lightweight deployment stack.

The project achieved its primary learning objective. The developer gained direct, hands-on experience with every stage of the pipeline and developed practical intuition for the trade-offs between dataset quality, model scale, retrieval design, and compute constraints that no amount of reading alone can provide.

The system, as it stands, is not production-ready. It makes factual errors, exhibits retrieval failures, and has been tested only informally. These limitations are honest reflections of what is achievable in a time-bounded personal project on free-tier compute — not embarrassments, but engineering data points.

The architecture is sound. The pipeline is end-to-end functional. The lessons learned about the interplay between RAG and fine-tuning, the brittleness of quantization toolchains, the centrality of data quality, and the difficulty of reliable evaluation are directly applicable to production AI engineering work. That was the goal, and it was met.